Week 3 Reference Sheet: String Operations & Debugging

CSC-125-201 | Use this as a reference when completing Homework 3

Homework Topics

- String methods: `strip()`, `upper()`, `lower()`, `title()`, `replace()`
- Method chaining
- String indexing with positive and negative indexes
- String slicing with `[start:end]`
- F-string formatting
- Reading and debugging error messages

Quick Reference

String Methods

Strings have built-in methods that return a new modified copy of the string. The original string is not changed.

```
message = "  Hello, World!  "

message.strip()      # "Hello, World!"   — removes leading/trailing whitespace
message.upper()      # "  HELLO, WORLD!  "
message.lower()      # "  hello, world!  "
message.title()      # "  Hello, World!  "

name = "alice smith"
name.upper()         # "ALICE SMITH"
name.lower()         # "alice smith"
name.title()         # "Alice Smith"

greeting = "Hello, World!"
greeting.replace("World", "Python")   # "Hello, Python!"
greeting.replace("l", "L")           # "HeLLo, WorLd!"
```

Method Chaining

Since each method returns a new string, you can call multiple methods in a row. Each method operates on the result of the previous one.

```
user_input = "   jOhN DOe   "

# Chain strip() and title() together
cleaned = user_input.strip().title()
print(cleaned)   # "John Doe"

# Chain multiple methods
text = "  hello world  "
result = text.strip().upper().replace("WORLD", "PYTHON")
print(result)    # "HELLO PYTHON"
```

String Indexing

Each character in a string has a position (index). Positive indexes count from the left starting at 0. Negative indexes count from the right starting at -1.

```
word = "Python"
#        P  y  t  h  o  n
#        0  1  2  3  4  5    (positive indexes)
```

```
#       -6 -5 -4 -3 -2 -1   (negative indexes)

word[0]     # "P"   — first character
word[1]     # "y"   — second character
word[5]     # "n"   — last character
word[-1]    # "n"   — last character
word[-2]    # "o"   — second to last

# len() returns the number of characters
len(word)   # 6
```

String Slicing

Slicing extracts a portion of a string using `[start:end]`. The start index is included, but the **end index is NOT included**.

```
word = "Python"
#        P  y  t  h  o  n
#        0  1  2  3  4  5

word[0:3]   # "Pyt"   — index 0, 1, 2 (stops before 3)
word[2:5]   # "tho"   — index 2, 3, 4 (stops before 5)

# Omit start — defaults to 0 (beginning)
word[:4]    # "Pyth"  — same as word[0:4]

# Omit end — defaults to end of string
word[2:]    # "thon"  — index 2 through the end

# Omit both — copies the entire string
word[:]     # "Python"

# Negative indexes work with slicing too
word[-3:]   # "hon"   — last 3 characters
word[:-2]   # "Pyth"  — everything except last 2
```

Finding Positions

The `.index()` method returns the position of the first occurrence of a substring. This is useful for dynamic slicing.

```
email = "student@college.edu"

# Find the position of the @ symbol
at_pos = email.index("@")
print(at_pos)          # 7

# Use .index() with slicing to extract parts
username = email[:at_pos]
print(username)        # "student"

domain = email[at_pos + 1:]
print(domain)          # "college.edu"
```

F-String Formatting

F-strings let you embed expressions directly inside string literals by prefixing the string with `f` and placing expressions inside curly braces `{}`.

```
name = "Alice"
age = 20
```

```python
# Basic variable insertion
print(f"Hello, {name}")           # "Hello, Alice"

# Expressions inside curly braces
print(f"Next year: {age + 1}")    # "Next year: 21"

# Method calls inside curly braces
print(f"Name: {name.upper()}")    # "Name: ALICE"
print(f"Name: {name.lower()}")    # "Name: alice"

# Decimal formatting with :.2f (2 decimal places)
price = 9.5
print(f"Total: ${price:.2f}")     # "Total: $9.50"

tax = 0.7825
print(f"Tax: ${tax:.2f}")         # "Tax: $0.78"
```

Common Errors

When your code produces an error, Python tells you the type of error and where it occurred. Here are common errors you may encounter with strings.

Error	Cause	Fix
`IndexError`	Accessing an index that does not exist (e.g., `word[10]` on a 6-character string)	Check the string length with `len()` and make sure your index is within range
`AttributeError`	Calling a method that does not exist on the data type (e.g., `num.upper()` on an integer)	Verify the variable is a string before calling string methods
`NameError`	Using a variable that has not been defined yet or is misspelled	Check for typos in variable names and make sure the variable is assigned before use
`TypeError`	Performing an unsupported operation between types (e.g., `"age: " + 20`)	Use `str()` to convert non-strings, or use an f-string instead

Additional Resources

Corey Schafer - Python Strings Tutorial

https://www.youtube.com/watch?v=k9TUPpGqYTo

- Video tutorial covering string basics, methods, and formatting

W3Schools - Modify Strings

https://www.w3schools.com/python/python_strings_modify.asp

- Reference for upper, lower, strip, and replace methods

W3Schools - Slicing Strings

https://www.w3schools.com/python/python_strings_slicing.asp

- Indexing and slicing syntax with examples

W3Schools - String Formatting

https://www.w3schools.com/python/python_string_formatting.asp

- F-string syntax and formatting options

Real Python - F-Strings Guide

https://realpython.com/python-f-strings/

- Detailed f-string examples and advanced usage

LearnPython.org - Basic String Operations

https://www.learnpython.org/en/Basic_String_Operations

- Interactive exercises for practicing string operations